

Class Exercise 1 - Real Estate Server

- Due Wednesday by 2:59am
- Points 20
- Available after May 11 at 10am

Real Estate Server Project

Aim: This course exercise will take you thru a series of short daily exercises that illustrate how software systems can grow and change over time. In many ways the 'art and science' of software engineering is learning methods and techniques that enable you to accomodate and manage these inevitable changes in the most effective, efficient and least costly manner.

These are small team exercises so one of your challenges will be to divide the tasks up so that they can be tackled in parallel to make yor team most efficient.

Task 1 - Get the starter code

Clone (git clone <repo> <directory>) and build (mvn clean install) the code from these two repos:

<https://github.com/gortonator/REServer> ➞ (<https://github.com/gortonator/REServer>): A **Javelin-based** ➞ (<https://javalin.io/>) HTTP server that offers a skeleton implementation of an application that creates and serves real estate sales data from NSW

<https://github.com/gortonator/REDataLoader> ➞ (<https://github.com/gortonator/REDataLoader>): A simple CSV file reader that inputs and displays all the records from an archive of NSW home sales.

If you use IntelliJ then you can create a new Maven project from existing sources and point it at pom.xml file for the project. Other IDEs you are on your own!

Task 2 - Test the Server

Run the real estate server then go to <http://localhost:7070/> ➞ (<http://localhost:7070/>) to test you application is alive. If it is, look at the code and figure out how to call the other GET and POST HTTP endpoints the server supports. To test the POST endpoint you'll need to use a tool such as **Postman** ➞ (<https://www.postman.com/downloads/>). If no one in your team is familiar with this, talk to your instructor.

Note there are no PUT or DELETE APIs as real estate sales data in immutable. It ain't CRUD!

Task 3 - Test the Data Loader

You'll need to point this code at the input data file that you can find **here** (<https://northeastern.instructure.com/courses/255294/files/41420911?wrap=1>) ↓ (https://northeastern.instructure.com/courses/255294/files/41420911/download?download_frd=1) . Running the code simply displays every record in the .csv file. As you'll see, it's quite large

Task 4 - Load a database

This is where the fun starts. Your task is to:

1. Choose and install a database to store the real estate data. Any database. Your choice.
2. Design and implement a data model that enables you to store the content of the real estate data input file in the database
3. Modify the data loader to write the records into your database. Loading databases with lots of data can be tricky - check the database docs to see what it recommends to achieve fast (bulk) loading.
4. Connect the REServer to your database and implement queries that return data from the APIs. You might find [this article](https://medium.com/@connectwithpradyumna/building-a-simple-crud-application-with-javalin-and-mysql-e58bdff585be)  (<https://medium.com/@connectwithpradyumna/building-a-simple-crud-application-with-javalin-and-mysql-e58bdff585be>) useful - steal code liberally if you like!
5. What problems do these APIs have? Provide a 'hack' so that you can call the APIs even though they don't return all the data that satisfies the API semantics.

Task 5 - Demo your work to an instructor

You're done.

Grading

1. Successfully loading the database - 10 points
2. Connecting REServer to the database and demo-ing APIs - 8 points
3. Completion in class - 2
4. First team to complete - 2 bonus points